

Graph RAG, in Practice

How it ingests, how it retrieves, how it compares to vanilla RAG, and a close look at LightRAG

What Graph RAG is

Vanilla RAG chunks documents, embeds each chunk as a vector, and retrieves the chunks nearest to your question. **Graph RAG** adds a structured layer: it extracts **entities** (nodes) and **relationships** (edges) into a knowledge graph and retrieves by traversing that structure. This buys two things vanilla RAG struggles with — multi-hop reasoning that links scattered evidence, and corpus-wide “what are the themes” questions — at the cost of heavier indexing and maintenance.

1 · The intake pipeline

Graph RAG front-loads work at indexing time. The two coral stages below each call an LLM for every chunk or cluster — that asymmetry versus vanilla RAG’s “chunk + embed” drives nearly all of the cost and maintenance differences later.

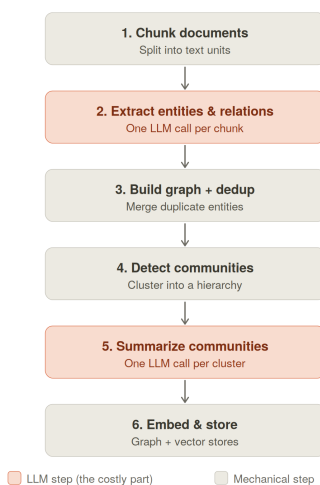


Figure 1 · Graph RAG intake. Only the coral steps use an LLM.

2 · Retrieval: a fork, not a blend

A query travels down *one* of two routes. **Local search** anchors on specific entities and walks the graph outward — best for “what is X and what connects to it?” **Global search** skips traversal and map-reduces over the community summaries built during intake — best for “what are the themes across everything?” Picking the wrong route is the most common cause of a disappointing answer: a specific question sent to global search returns a vague summary.

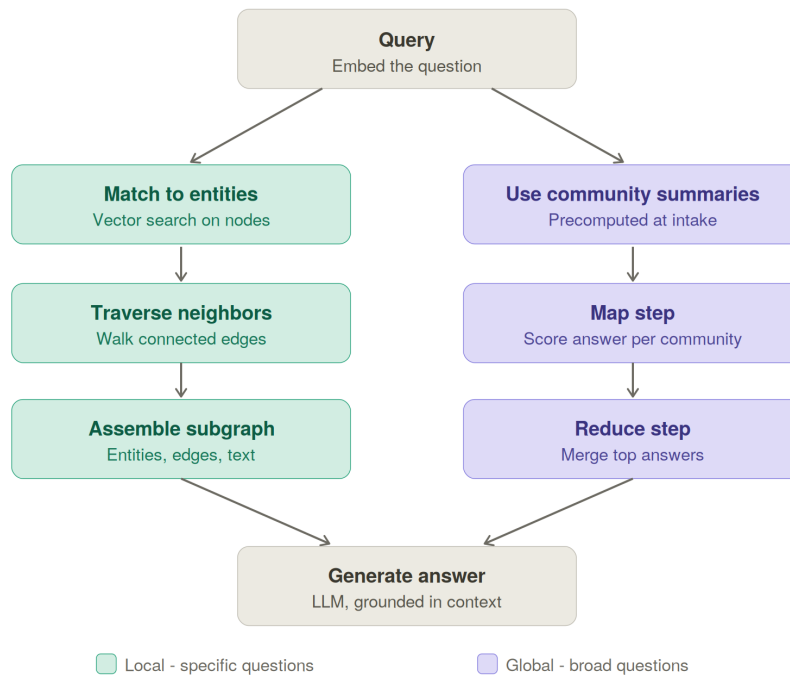


Figure 2 · The two retrieval modes share a start and end but run separately.

3 · Performance vs. vanilla RAG

Recent benchmarks are pointed: Graph RAG's advantage is **task-dependent, not universal**, and on many ordinary lookups it matches or trails vanilla RAG. The split:

Task type	Who tends to win	Why
Single-hop / factual lookup	Vanilla RAG (or tie)	Graph extraction can lose or distort precise detail.
Multi-hop / reasoning	Graph RAG, often by a lot	Links evidence scattered across documents via typed edges.
Corpus-wide summarization	Graph RAG, large margin	Community summaries synthesize the whole corpus at once.

By metric. *Recall/precision* are comparable on simple lookups; Graph RAG pulls ahead when evidence is scattered and relational. *Accuracy* swings on task type (big multi-hop gains, marginal on simple QA). *Faithfulness* can improve via traceable relationship paths, but inherits a new failure mode — a wrong extraction at intake gets faithfully reproduced. A well-tuned vanilla pipeline with agentic search narrows much of the gap, so treat these as directional, not guarantees.

4 · Cost, maintenance & speed

Axis	Graph RAG vs. vanilla RAG
Indexing cost	Much higher — LLM extraction per chunk plus LLM summarization per cluster.

Query-time cost	Mixed. Global queries can be cheaper (pre-baked summaries); local/hybrid often costlier.
Maintenance	Heavier — a graph store plus extraction & clustering pipelines; updates may force re-clustering.
Speed	Ingestion much slower; query latency usually higher but variable.

Rule of thumb: Graph RAG earns its overhead roughly past ~100K documents, or when multi-hop / cross-cutting questions are routine (compliance, contract analysis, diagnostics). Below that, vanilla RAG — optionally with reranking — is usually the better engineering bet.

5 · Community summaries are sufficient, not required

Microsoft's community-detection-plus-summary approach made global sensemaking famous, but it is one method, not the requirement. What's actually needed is some way to **compress the whole corpus** into a form an LLM can reason over. Other routes reach the same goal — and this is exactly why LightRAG can skip community detection entirely.

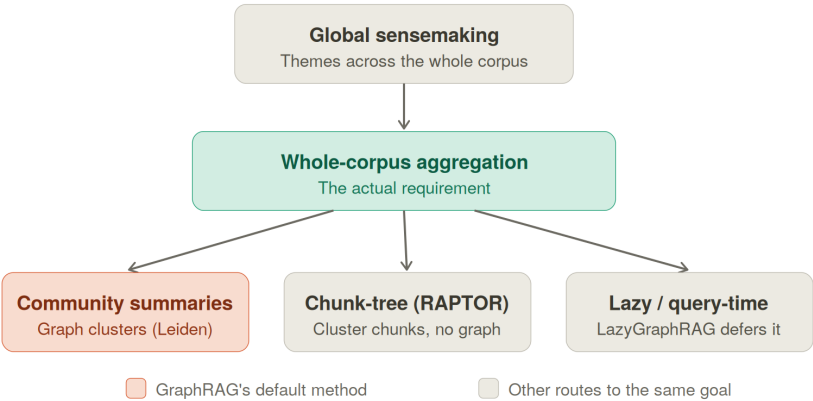


Figure 3 · The requirement is aggregation; community summaries are just one way there.

6 · The library landscape

Library	Best for	Worth knowing
Microsoft GraphRAG	Global sensemaking	Reference implementation; famously expensive to index.
LightRAG	Practical default	Cheap indexing, incremental updates, strong momentum.
Graphiti	Agent memory over time	Temporal edges; needs Neo4j.
nano-graphrag	Learning / hacking	Minimal, easy-to-modify implementation.
neo4j-graphrag	Neo4j-committed teams	First-party; a toolkit of retrievers (incl. Text2Cypher), not a turnkey global pipeline.

Stars track buzz, not fit — benchmark numbers are close and dataset-dependent, so evaluate on your own data.

7 · Spotlight: how LightRAG works

LightRAG's whole bet is to push as little as possible into intake, then recover breadth at query time. Its intake drops the two priciest Graph RAG stages — community detection and per-cluster summarization — leaving a single LLM step. That alone is most of its cost and incremental-update advantage.

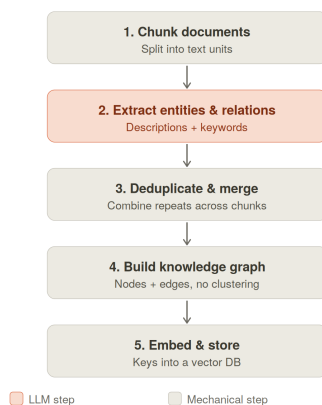


Figure 4 · LightRAG intake: no clustering, no community summaries.

At query time it runs **dual-level retrieval** — both levels together, every query, with no local-vs-global fork. An LLM splits the query into **low-level** keywords (specific, matched against entity nodes) and **high-level** keywords (broad, matched against relationship edges). Because every edge carries its own description and keywords, that edge layer holds the thematic information GraphRAG would have baked into community summaries — retrieved on demand instead of pre-computed.

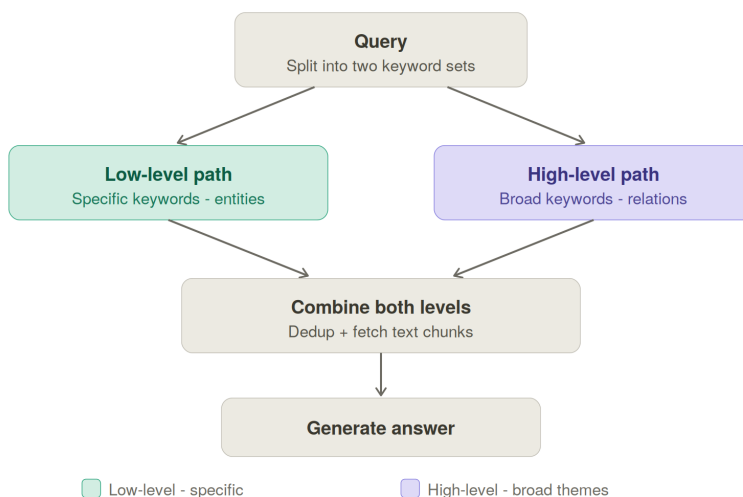


Figure 5 · Dual-level retrieval runs both paths, then merges.

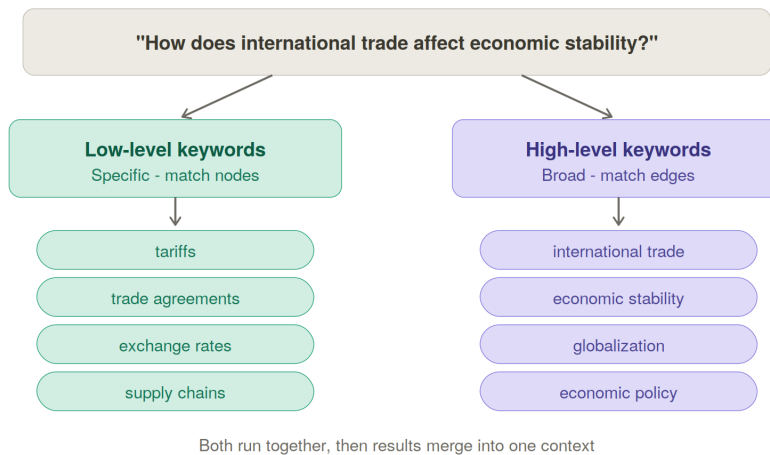


Figure 6 · One query, split two ways. Low-level matches nodes; high-level matches edges.

How disconnected matches resolve. The node set and edge set need not connect. Each match is first expanded into its local neighborhood, then everything is pooled and deduplicated — no join, no path-finding. When both paths land on the same node, dedup quietly fuses the two regions into one. Whether that shared node *exists* was decided earlier by entity resolution at intake; if that step under-merges a real-world entity, the regions stay split even when they should connect.

Bottom line

Reach for vanilla RAG for single-fact lookups at modest scale. Reach for Graph RAG when answers depend on **linking dispersed evidence** or **synthesizing across a whole corpus**. Among graph tools, Microsoft GraphRAG sets the quality bar for global questions, while LightRAG is the pragmatic default — most of the benefit, a fraction of the indexing cost, and the cheap updates that matter in production.